Hierarchical Models and Partial Pooling

ECON 8310: Business Forecasting — Lecture 11, Part 2

Department of Economics
University of Nebraska at Omaha

August 25, 2026

Lecture Outline

Thirty Series, Two Bad Options

You have done this the wrong way twice already. Here is the third way.

The Question

A concrete one, and you have met it before. **Does demand rise in weeks when SNAP benefits are disbursed, and by how much — for each of our thirty store-category series?**

Homework 3's memo asked you to decide which store-categories to stock up for a SNAP week rather than applying one national uplift. That question assumed the per-series differences are real. Today we check.

The complication is that the series are not equally informative. Some have years of history; in the real world some are new stores, new products, or recently re-categorized, with only a handful of usable weeks.

> **The setup for today.** We keep all thirty series, but deliberately thin every third one down to **8 weeks** of data. Twenty series are data-rich; ten are data-poor. That imbalance is the whole point.

Complete Pooling and No Pooling

There are two obvious approaches, and each fails in a different direction.

	What it does	How it fails
Complete pooling	One SNAP effect for all thirty series. `b = pm.Normal("b", 0, 1)`	Denies that stores differ at all. A genuinely different store is averaged away.
No pooling	A separate effect per series, estimated independently. `shape=30`	A series with 8 weeks gets an estimate built from 8 weeks. Mostly noise, reported as a finding.

You have used both already without naming them. Homework 3's random forest pooled all thirty series into one model with store dummies — close to complete pooling. Homework 1 fitted ARIMA to one series at a time — no pooling.

Neither is obviously right, and the choice has always been made by convenience. Partial pooling makes it a modelling decision instead, and lets the data settle it.

Partial Pooling

Let each series have its own effect — drawn from a shared distribution the data also estimates.

The Hierarchy Is One Extra Line

Give each series its own effect b_j, but say where those effects come from:

$$b_j \sim \mathcal{N}(\mu_b,\ \sigma_b), \qquad j = 1, \ldots, 30$$

μ_b is the typical SNAP effect across the business, and σ_b is **how much stores genuinely differ**. Crucially, both are estimated from the data rather than assumed.

$\sigma_b \to 0$	The model concludes stores do not differ, and reproduces complete pooling.
σ_b large	The model concludes they differ a lot, and approaches no pooling.
In between	Each b_j is pulled toward μ_b — *how far* depending on how much data that series has.

That last row is the whole idea. A series with 277 weeks barely moves; a series with 8 weeks moves a long way, because its own estimate carries little information. Nobody chose those weights — they fall out of the model.

Why This Is the Right Answer, Not a Compromise

Partial pooling sounds like splitting the difference between two bad options. It is better than that, and the reason is worth stating.

When a series has 8 observations, its unpooled estimate is mostly sampling noise. The hierarchy's pull toward μ_b is not a fudge — it is the correct response to a noisy measurement. You already believe this in another form: it is why you would not conclude a coin is biased after three flips.

> **This is regularization again, for the sixth time.** A hierarchical prior shrinks per-group estimates toward a common value exactly as Ridge shrinks coefficients toward zero — and here the shrinkage strength σ_b is *estimated* rather than tuned by cross-validation.

GAM smoothing penalty (L3) → tree pruning (L4) → `reg_lambda` (L5) → Ridge and LASSO (L6) → weight decay (L7) → hierarchical priors (today).

Fitting It, and the Trap

The obvious code does not work, and the reason is geometry rather than statistics.

The Model in PyMC

Non-centred hierarchical model

```
with pm.Model() as model:

mu_b = pm.Normal("mu_b", 0, 0.5)       # typical SNAP effect

sd_b = pm.HalfNormal("sd_b", 0.25)     # how much stores differ

z    = pm.Normal("z", 0, 1, shape=J)    # non-centred

b    = pm.Deterministic("b", mu_b + z * sd_b)

a = pm.Normal("a", y.mean(), 1, shape=J)  # per-series level

s = pm.Exponential("s", 1.0)

pm.Normal("obs", mu=a[idx] + b[idx]*snap, sigma=s, observed=y)
```

`idx` maps each row to its series, so `b[idx]` picks out the right effect for every observation. That indexing pattern is how every hierarchical model in PyMC is written.

Note what is *not* here. No decision about how much to pool, no weighting scheme, no threshold for "enough data." Those all come out of estimating σ_b.

The Trap: Write It the Obvious Way and It Breaks

The natural way to write the hierarchy is `b = pm.Normal("b", mu_b, sd_b, shape=J)` — the *centred* form, which reads exactly like the equation. On our data it produces:

	Centred	**Non-centred**	**Want**
Divergences	**104**	**0**	0
$\hat{R}$	1.030	1.000	< 1.01
ESS (bulk)	—	3,846	> 400

Why: when σ_b is small, the b_j must crowd into a narrow band, and the posterior takes on a funnel shape — wide where σ_b is large, pinched where it is small. A sampler tuned for the wide part cannot navigate the neck, and reports divergences.

The non-centred form samples a standard Normal z and *builds* $b_j = \mu_b + z_j \sigma_b$ afterwards. Same model, same answer, geometry the sampler can walk.

This is BMCP §4.6.1, Posterior Geometry Matters, and it is the single most common failure in applied hierarchical modelling.

Reading the Result

Shrinkage, measured — and a business conclusion that reverses the obvious one.

Shrinkage, Measured

Fit both models. Compare each series' estimated SNAP effect before and after pooling.

Series group	Unpooled spread	After pooling	Mean move
Thin — 8 weeks (10 series)	0.106	0.000	**0.110**
Full — 277 weeks (20 series)	0.012	0.001	0.010

The data-poor series moved 11.5 times further toward the group mean than the data-rich ones. Nobody told the model which series were thin. It inferred that from how much each series had to say.

Look also at the unpooled column. The thin series appear to vary nine times more than the full series (0.106 against 0.012). They are the *same kind* of store-category — that extra spread is not business variation, it is what 8 observations look like.

The hierarchy did not average away a real signal. It removed a fake one.

What the Model Actually Concluded

The estimated hierarchy: $\mu_b = 0.011$, $\sigma_b = 0.007$.

In words: the typical SNAP-week uplift is about **1%** on log units, and the genuine spread across store-categories is smaller than that again. The model has concluded the thirty series are, for this purpose, **nearly the same**.

> **That reverses the natural business instinct.** An analyst reading the unpooled fit would report SNAP uplifts ranging from roughly -2% to $+35\%$, and might well build a store-specific stocking policy on it. Almost all of that range was noise from the thin series.

So the honest recommendation is the boring one: apply a common uplift, and do not differentiate by store-category until more data justifies it. That is a real finding, it is worth money, and no point-estimate procedure would have told you.

Weekly aggregation is part of the story — the daily SNAP effect in Homework 2 was larger, because most weeks contain some SNAP days.

Key Takeaways

What hierarchy buys, and what it costs.

Lecture 11 Part 2: Key Takeaways

1. **Complete pooling** denies groups differ; **no pooling** lets a group with 8 observations report noise as a finding. You have used both without naming them.
2. **Partial pooling** gives each group its own parameter drawn from a shared distribution, and estimates how much groups genuinely differ.
3. **Shrinkage is automatic and proportionate.** Our thin series moved $\mathbf{11.5}\times$ further toward the group mean than data-rich ones, with nobody specifying that.
4. This is **regularization for the sixth time** — and the only version where the shrinkage strength is estimated rather than tuned.
5. **Write it non-centred.** The obvious form gave 104 divergences; the reparameterized form gave zero. Same model, different geometry.
6. The business conclusion **reversed the obvious one**: apparent store-level differences were mostly noise, and a common uplift is the defensible policy.

Next: Bayesian linear regression — coefficients as distributions, and what you can ask them.

References I